

The Bitter Lesson of Tool Calling

Ishan Patel, Sahil Sen, Elias Lumer, and Vamse Kumar Subbiah

Commercial Technology and Innovation Office

PricewaterhouseCoopers, U.S.A

Abstract

Tool use transforms LLMs into agents that act beyond their training data, and for code-capable models, programmatic tool calling extends this further by replacing rigid JSON calls with scripts that chain and parallelize naturally. However, a systematic evaluation of tools as code on an established benchmark across current and prior model generations under real-world task conditions has not been conducted. In this work, we empirically compare programmatic tool calling (PTC) to native JSON tool calling across 14 language models on BFCL v4. In the programmatic tool calling paradigm, tools are exposed as typed Python stubs that the model invokes through code, with execution and results handled in a single agent turn. Programmatic tool calling matches or exceeds native JSON tool calling in 11 of 14 models on BFCL v4, with the GPT-5.6 family achieving a 10.6% improvement over the JSON tool calling baseline. Further, it matches or outperforms baseline in 13 of 14 models under parallel fan-out, and holds stable under context rot conditions where baseline degrades 2.3% on average. Our results demonstrate that programmatic tool calling is a viable and robust alternative to JSON tool calling, with performance tracking model capability across release generations.

1 Introduction

Large language models (LLMs) now act as tool-calling agents, invoking external services through APIs that require the model to emit a structured JSON object at each function call (Anthropic, 2024; Guo et al., 2026). For models that can already write executable code, this format is a design choice, not a necessity.

Prior work has established a theoretical case for replacing JSON tool calls with executable code. CodeAct (Wang et al., 2024) showed that code actions achieve up to 20% higher task success with

30% fewer interaction turns on multi-tool tasks, with gains concentrated in parallel and compositional scenarios. A more recent study found that code-only output restrictions change pass rates by an absolute change of fewer than 3% on coding-agent tasks (Yang et al., 2026). Function calling is a harder test case. It requires precise argument serialization, multi-step chaining, and fan-out across heterogeneous APIs, constraints absent from coding-agent evaluations and exactly where paradigm choice has the largest claimed effect. Whether programmatic tool calling matches JSON tool calling under these conditions, across model families and adversarial context loads, remains untested.

To address this gap, we evaluate *programmatic tool calling* against JSON tool calling on a representative 309-entry subset of BFCL v4 spanning eight task categories, using 14 models released between November 2024 and July 2026. In programmatic tool calling, the model writes a Python script using typed Python stubs compiled from the benchmark’s function schemas. The agent loop executes it in a shell subprocess, producing tool-call results without additional inference turns. We run three ablation studies targeting sequential chaining, parallel fan-out, and context rot, the task structures where the limitations of JSON tool calling have been most commonly claimed.

We find that 11 of 14 models match or exceed baseline accuracy under programmatic tool calling on BFCL v4, with results dividing along model generation lines. All five Anthropic models and the three newest GPT generations match or exceed baseline accuracy, while three older GPT models do not. On context flooding, programmatic tool calling sees an absolute improvement of 5.5% on average, compared to an absolute 32% degradation for the filesystem-discovery comparison approach.

Our contributions are:

- Programmatic tool calling viability divides along model generation lines rather than model family. All five Anthropic models and the three newest GPT generations match or exceed JSON tool calling accuracy on BFCL v4, while three older GPT models do not.
- Programmatic tool calling’s accuracy advantage on sequential tasks scales with chain length, reaching an 18.8% absolute gap over JSON tool calling at lengths ≥ 12 , an effect absent at short chains and driven by the extra inference turn JSON tool calling incurs per link.
- JSON Tool Calling drops tool calls entirely above a model-specific fan-out threshold ($N = 70\text{--}72$ for Claude Sonnet 5). Programmatic tool calling maintains 100% enumeration accuracy at $N = 100$, exposing a hard structural limit in the native paradigm.
- Under context flooding, programmatic tool calling holds stable while JSON tool calling degrades 2.3% on average and the filesystem-discovery approach degrades 32%, demonstrating robustness to adversarial context load.

2 Related Work

Two lines of work motivate replacing JSON tool calls with executable code. The first is code-action agents. CodeAct (Wang et al., 2024) showed that code actions outperform JSON alternatives on multi-tool tasks, with gains concentrated in parallel and compositional scenarios. Recursive Agent Harnesses (Lumer et al., 2026) extends the same code-execution primitive to parallel subagent spawning, showing that executing code in a subprocess bypasses the per-turn tool-call cap that constrains native JSON function calling at high fan-out. Practitioner frameworks extended this finding into production, treating code composition as the default interface (Hugging Face, 2024; Anthropic, 2024; Karten et al., 2026). Production memory systems such as Chronos (Sen et al., 2026b) deploy iterative JSON tool-calling loops as their core agent mechanism, confirming that JSON tool calling remains the dominant interface in deployed agentic systems. Infrastructure providers are adopting the same pattern. Cloudflare’s Agents platform ships code execution as its native tool-use interface (currently experimental) (Cloudflare, 2026). A recent survey

argues that code is the ideal substrate for agentic reasoning because it is executable, inspectable, and stateful (Ning et al., 2026). The Deterministic Horizon (Guo et al., 2026) provides theoretical grounding. Tool delegation is necessary precisely where chain-of-thought reasoning fails to produce verifiable answers, the same regime where tool-calling accuracy is measured. However, none of these works evaluate code-execution against JSON tool calling on a standardized benchmark across multiple model families.

Several benchmarks measure LLM tool-calling accuracy, including API-Bank (Li et al., 2023), T-Eval (Chen et al., 2024), API-BLEND (Basu et al., 2024), UltraTool (Huang et al., 2024), CONFETTI (Alkhoul et al., 2025), and ToolHop (Ye et al., 2025). Each evaluates how accurately models call the right function with the right arguments, but none compares paradigms. BFCL v4 covers eight task categories with a deterministic scorer. A recent audit (Vaghasiya et al., 2026) found 20% evaluator-human misalignment in its LLM-judge mode, which our evaluation sidesteps by scoring against stub outputs directly. Work on agentic tool-calling training efficiency (Liu et al., 2026) and context inflation (Du et al., 2026) identifies format and prompt engineering as dominant sources of variance in tool-calling benchmarks, motivating the controlled three-way paradigm comparison we conduct. Unlike prior work, we evaluate programmatic tool calling against JSON tool calling on BFCL v4 across 14 models spanning 20 months of releases, with three ablation studies targeting chaining, parallel fan-out, and context rot.

3 Method

We study whether programmatic tool calling can replace JSON tool calling for LLM tool use without sacrificing accuracy. We evaluate two paradigms on BFCL v4, *JSON tool calling* and *programmatic tool calling*. In each paradigm, the model receives the same task description and must invoke the correct functions with the correct arguments. Only the mechanism for expressing and executing tool calls differs. In the context rot ablation we additionally include a filesystem-based condition as an internal reference point. It is not an industry practice and is referenced only where it illuminates the primary comparison.

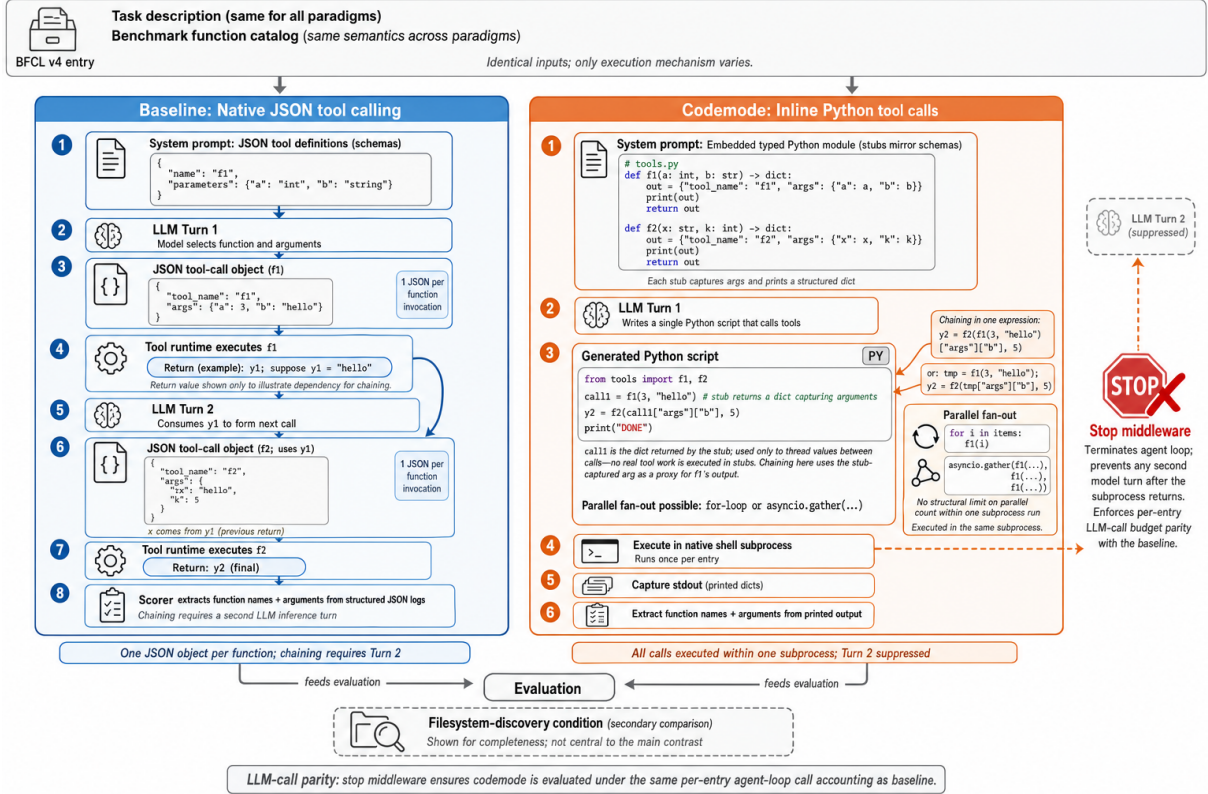


Figure 1: Overview of the two primary paradigms evaluated. In *JSON tool calling*, the model emits JSON tool-call objects via the API. In *programmatic tool calling*, it writes a Python script using typed stubs; the agent loop executes it in a subprocess. A filesystem-discovery condition is included as a secondary reference point.

3.1 Task Definition

Each benchmark entry specifies a natural-language user query q , a set of available functions $\mathcal{F} = \{f_1, \dots, f_k\}$ each with a typed signature, and a ground-truth set of function calls $\mathcal{C}^* = \{(f_i, \mathbf{a}_i)\}$ where $\mathbf{a}_i$ is the argument mapping for call i . A model is correct on an entry if and only if its output produces a set of calls that matches $\mathcal{C}^*$ under the benchmark’s normalized string comparison, which strips punctuation and lowercases all values before matching. This definition is identical across all three paradigms. Differences arise only in how the model is prompted and how its output is parsed.

3.2 Paradigms

JSON Tool Calling. The model receives a system prompt containing the function schemas formatted as JSON tool definitions and an API call that invokes the tool-calling endpoint. The model outputs a structured JSON object at each function call. This is the standard deployment pattern for tool-augmented LLMs and serves as our reference condition.

Programmatic tool calling. The model receives a system prompt that embeds the source of a typed Python module whose functions correspond one-to-one with the benchmark’s function schemas. Each stub function captures its arguments and returns them as a structured dict. No external service is contacted. The model writes a Python script that imports this module and calls the appropriate functions. The agent loop executes the script in a native shell subprocess. The subprocess captures stdout, and the scorer extracts function names and argument values from the printed output. No additional inference turns occur after the subprocess returns. A stop middleware intercepts the next model call and terminates the agent loop. This design ensures that programmatic tool calling and JSON tool calling consume the same number of LLM calls per entry, making accuracy comparisons straightforward.

Consider a task that asks the model to retrieve the circumference of a circle with radius 7 and the area of a square with side 5. In JSON tool calling, the model emits two sequential JSON tool-call objects. In programmatic tool calling, the model writes:

```

execute(command="python3 -c '
import json
from stubs import (
    circumference, area_square)
print(json.dumps(circumference(radius=7)))
print(json.dumps(area_square(side=5)))
'")

```

The shell subprocess captures both lines of std-out, and the scorer matches each printed call against the ground truth. Programmatic tool calling resolves both calls in a single Python expression evaluated in one subprocess, while JSON tool calling requires two separate model outputs.

3.3 Benchmark and Evaluation

BFCL v4 (Vaghisiya et al., 2026) provides 309 representative entries drawn from eight task categories: *simple_python*, *multiple*, *parallel*, *parallel_multiple*, *live_simple*, *live_multiple*, *live_parallel*, and *live_parallel_multiple*. Entries were sampled proportionally to mirror the category distribution of the full BFCL v4 benchmark, with per-category minimums applied to ensure sufficient entries in smaller categories. The complete entry ID list is released with the evaluation harness. Simple and multiple categories test single-call and multi-call accuracy on curated queries. Live categories use queries sampled from real user interactions. Parallel categories require the model to issue two or more function calls whose arguments are independent of each other.

All three paradigms use a deterministic scorer that normalizes predicted and ground-truth argument values before comparison. We report accuracy as the fraction of entries where every required function call is both present and correctly parameterized. When the shell subprocess raises a syntax or runtime error, the entry is scored as zero correct calls. No entries are skipped or excluded from reported totals.

3.4 Ablation Design

The three ablation studies target task structures where paradigm choice has been claimed to matter. Each ablation subset was constructed by selecting entries from BFCL v4 that represent the target task structure without duplicating main-evaluation entries. The complete entry ID lists are released with the evaluation harness. The chaining subset ($n = 52$) was optimized to cover the full range of chain lengths ($n_{\text{chain}} = 2\text{--}20$), with the distribution weighted toward longer chains where JSON tool calling and programmatic tool calling

diverge most ($n_{\text{chain}} \geq 6$: 17 entries; $n_{\text{chain}} \geq 13$: 10 entries). The parallelism subset ($n = 32$) covers fan-out counts of 7 to 48 (7, 9, 11, 13, 15, 20, 30, 48), drawn from a synthetic corpus. We report only enumeration-type entries ($n = 32$ of 48 total), excluding aggregation-type entries from the accuracy figures. To locate the fan-out threshold at which Anthropic frontier models begin dropping calls under JSON tool calling, we extend the corpus with probe entries at $N \in \{60, 70, 72, 75, 100\}$ and evaluate Claude Sonnet 5 baseline and programmatic tool calling on those entries. The context rot subset ($n = 31$ per condition) uses entries from *live_multiple*, *live_parallel*, and *live_parallel_multiple* categories where function schemas are most easily contaminated by domain-unrelated decoys.

Chaining. Sequential multi-hop function calls, where the output of f_1 must be computed and passed as an argument to f_2 . In JSON tool calling, this requires two model turns. The model calls f_1 , receives the return value, then calls f_2 . In programmatic tool calling, both calls appear in a single script. The model computes the intermediate value using parametric knowledge and passes it directly. The chaining ablation contains 52 entries with chain lengths of 2 to 20 calls.

Parallelism. Independent fan-out function calls, where N calls must be issued in a single step. JSON tool calling issues them as parallel tool-call objects. Programmatic tool calling writes an `asyncio.gather` block or a sequential loop. The parallelism ablation uses 32 enumeration-type entries with fan-out counts ranging from 7 to 48, plus targeted probe entries at $N \in \{60, 70, 72, 75, 100\}$ for Claude Sonnet 5 to locate the fan-out threshold at which JSON tool calling begins dropping calls. We separately report *enumeration accuracy* (did the model issue all N required calls?) and *aggregation accuracy* (did the model produce the correct aggregate answer?), as programmatic tool calling can answer aggregation questions from parametric knowledge without executing every call.

Context rot. The flood condition injects decoy function schemas into the context alongside the entry’s relevant schemas, increasing total context size to 128 schemas. The two conditions are *filtered* (only the functions the query uses) and *flood* (128 total schemas, including the relevant functions plus corpus decoys drawn from unrelated domains).

Name	Model ID
<i>Anthropic</i>	
Claude Haiku 4.5	claude-haiku-4-5
Claude Sonnet 4.5	claude-sonnet-4-5
Claude Sonnet 4.6	claude-sonnet-4-6
Claude Opus 4.8	claude-opus-4-8
Claude Sonnet 5	claude-sonnet-5
<i>OpenAI</i>	
GPT-4o	gpt-4o-2024-11-20
GPT-4.1	gpt-4.1-2025-04-14
GPT-5-nano	gpt-5-nano-2025-08-07
GPT-5	gpt-5-2025-08-07
GPT-5.4-mini	gpt-5.4-mini-2026-03-17
GPT-5.4	gpt-5.4-2026-03-05
GPT-5.6-Luna	gpt-5.6-luna
GPT-5.6-Sol	gpt-5.6-sol
GPT-5.6-Terra	gpt-5.6-terra

Table 1: Models evaluated, covering releases from November 2024 to July 2026.

The context rot ablation contains 31 entries per condition.

3.5 Models

We evaluate 14 models spanning two families and 20 months of releases, listed in Table 1. All models are run at temperature 0.

All tables report 95% Wilson confidence intervals per row. Because ablation entries are small ($n = 31\text{--}52$), intervals are wide and individual model results should be treated as directional rather than conclusive. Only cross-model aggregate patterns are interpreted as reliable findings. Pairwise per-model comparisons are not corrected for multiple testing. We make no claim that any individual model result is statistically significant in isolation.

4 Experiments

We evaluate 14 models on BFCL v4 and on three ablation studies (chaining, parallelism, and context rot). All experiments are run at temperature 0. The results below report accuracy as the fraction of entries where every required function call is both present and correctly parameterized.

4.1 BFCL v4 Main Evaluation

Table 2 reports accuracy for JSON tool calling and programmatic tool calling across all 14 models on BFCL v4. The GPT-5.6 family achieves the largest programmatic tool calling gains. GPT-5.6-Sol and GPT-5.6-Terra each achieve an absolute improvement of 10.6% over their own JSON tool calling baseline. Across all 14 models, 11 match or exceed baseline accuracy under programmatic tool

Model	JSON		PTC	
	Acc	$\pm$	Acc	$\pm$
<i>Anthropic</i>				
Claude Haiku 4.5	81.9	4.3	85.8	3.9
Claude Sonnet 4.5	86.4	3.8	87.7	3.7
Claude Sonnet 4.6	80.9	4.4	87.4	3.7
Claude Opus 4.8	84.8	4.0	84.8	4.0
Claude Sonnet 5	84.5	4.0	86.1	3.9
<i>OpenAI</i>				
GPT-4o	81.9	4.3	55.0	5.5
GPT-4.1	81.9	4.3	62.1	5.4
GPT-5-nano	66.7	5.2	68.3	5.2
GPT-5	71.5	5.0	76.1	4.7
GPT-5.4-mini	79.3	4.5	55.0	5.5
GPT-5.4	79.3	4.5	81.9	4.3
GPT-5.6-Luna	76.1	4.7	80.3	4.4
GPT-5.6-Sol	72.2	5.0	82.8	4.2
GPT-5.6-Terra	73.5	4.9	84.1	4.1

Table 2: Accuracy (%) on our 309-entry BFCL v4 subset. JSON = JSON tool calling; PTC = programmatic tool calling. $\pm$ columns show 95% Wilson CI half-widths. Bold indicates the higher value per row (ties bolded on both).

calling. Per-category mean accuracies across all 14 models are reported in Table 6 (Appendix). The aggregate result conceals meaningful category-level variance. Programmatic tool calling underperforms JSON tool calling by an absolute 14.1% on average in the *parallel* and *parallel_multiple* categories, while gaining an absolute 10.0% in *live_multiple*. The parallel-category gap is partially explained by the `\n` encoding failures of the three weaker OpenAI models, which are most harmful on multi-call scripts. Excluding those three models narrows the gap.

All five Anthropic models (Claude Haiku 4.5 through Sonnet 5) match or exceed baseline accuracy under programmatic tool calling, with deltas ranging from 0.0 to 6.5 percentage absolute, a pattern that holds across model generations. Among OpenAI models, the three newest GPT-5.6 variants are all positive (absolute improvements of 4.2% to 10.6% over baseline), while three older models (GPT-4o, GPT-4.1, and GPT-5.4-mini) fall below baseline by an absolute 19.7% to 26.9%. For those three models the failure mode is consistent. The models produce code with literal `\n` escape sequences in multiline scripts rather than real newlines, causing the subprocess to fail with a syntax error on any entry requiring more than a single-line script. GPT-5-nano, released alongside GPT-5, does not exhibit this failure, suggesting the fix entered training data between GPT-5.4-mini and

Model	JSON		PTC	
	Acc	$\pm$	Acc	$\pm$
<i>Anthropic</i>				
Claude Haiku 4.5	88.5	8.8	73.1	11.7
Claude Sonnet 4.5	90.4	8.2	88.5	8.8
Claude Sonnet 4.6	90.4	8.2	92.3	7.6
Claude Opus 4.8	80.8	10.6	94.2	6.8
Claude Sonnet 5	80.8	10.6	96.2	6.0
<i>OpenAI</i>				
GPT-4o	92.3	7.6	86.5	9.3
GPT-4.1 [†]	98.1	4.9	40.4	12.9
GPT-5-nano	69.2	12.2	80.8	10.6
GPT-5	92.3	7.6	92.3	7.6
GPT-5.4-mini	76.9	11.2	80.8	10.6
GPT-5.4	94.2	6.8	90.4	8.2
GPT-5.6-Luna	82.7	10.2	92.3	7.6
GPT-5.6-Sol	96.2	6.0	90.4	8.2
GPT-5.6-Terra	98.1	4.9	96.2	6.0

Table 3: Chaining ablation accuracy (% , $n = 52$, chain lengths 2–20). JSON = JSON tool calling; PTC = programmatic tool calling. $\pm$ columns show 95% Wilson CI half-widths. Bold indicates the higher value per row (ties bolded on both). [†]GPT-4.1 PTC collapse caused by `\n` encoding failure; see text.

GPT-5.

4.2 Chaining Ablation

In the chaining ablation ($n = 52$ entries, chain lengths 2–20), programmatic tool calling handles sequential multi-hop calls differently from JSON tool calling. In JSON tool calling, the model issues f_1 , receives its return value, then issues f_2 in a second inference turn. In programmatic tool calling, the model writes both calls in a single script and computes the intermediate value from parametric knowledge before passing it to f_2 .

Table 3 shows per-model results. Claude Sonnet 5 achieves the largest programmatic tool calling gain (80.8% $\rightarrow$ 96.2%), followed by Claude Opus 4.8 (80.8% $\rightarrow$ 94.2%), while six models maintain near-parity (within 5% absolute). GPT-4.1 is the sole outlier. Its baseline accuracy of 98.1% collapses to 40.4% under programmatic tool calling, driven by the `\n` encoding failure described above, which is most harmful in chaining tasks because every intermediate computation requires a multiline script.

4.3 Parallelism Ablation

Programmatic tool calling matches or exceeds baseline accuracy for 13 of 14 models on the parallelism ablation ($n = 32$ entries, fan-out counts of 7 to 48 (7, 9, 11, 13, 15, 20, 30, 48)). Table 4 separates

Model	JSON		PTC	
	Acc	$\pm$	Acc	$\pm$
<i>Anthropic</i>				
Claude Haiku 4.5	100.0	5.5	100.0	5.5
Claude Sonnet 4.5	100.0	5.5	100.0	5.5
Claude Sonnet 4.6	100.0	5.5	100.0	5.5
Claude Opus 4.8	100.0	5.5	100.0	5.5
Claude Sonnet 5	100.0	5.5	100.0	5.5
<i>OpenAI</i>				
GPT-4o	100.0	5.5	100.0	5.5
GPT-4.1	96.9	7.5	90.6	10.5
GPT-5-nano	78.1	14.0	87.5	11.5
GPT-5	71.9	14.5	96.9	7.5
GPT-5.4-mini	100.0	5.5	100.0	5.5
GPT-5.4	100.0	5.5	100.0	5.5
GPT-5.6-Luna	100.0	5.5	100.0	5.5
GPT-5.6-Sol	96.9	7.5	100.0	5.5
GPT-5.6-Terra	100.0	5.5	100.0	5.5

Table 4: Parallelism ablation accuracy (% , $n = 32$). JSON = JSON tool calling; PTC = programmatic tool calling. Enumeration and overall accuracy are identical for all models here; aggregation accuracy is discussed separately in the text. $\pm$ columns show 95% Wilson CI half-widths. Bold indicates the higher value per row (ties bolded on both).

enumeration accuracy (did the model issue all N required calls?) from aggregation accuracy (did the model produce the correct aggregate answer?).

The largest programmatic tool calling gain is GPT-5, which improves from 71.9% to 96.9%. For this model, baseline accuracy degrades as fan-out count increases above 13, while programmatic tool calling maintains near-perfect enumeration across all fan-out levels. The single model that falls below baseline under programmatic tool calling is GPT-4.1 (90.6% vs. 96.9% baseline), again attributable to the `\n` encoding failure on parallel scripts that require `asyncio.gather`.

We observe that programmatic tool calling enumeration accuracy is high while aggregation accuracy is lower and noisier across models. Inspection of outputs shows that models frequently produce the correct aggregation answer from parametric world knowledge (stating, for example, the top-3 countries by population directly) without executing the enumeration calls. This is a correct answer by the scorer’s standard, but it does not test whether the model actually executed the tools. We report both metrics separately and treat enumeration accuracy as the primary measure of paradigm fidelity.

Token costs cross over at $N \approx 26$. Below this threshold programmatic tool calling is more expensive due to its fixed system-prompt overhead.

Model	Filtered			Flood		
	JSON	PTC	Arg	JSON	PTC	Arg
<i>Anthropic</i>						
Haiku 4.5	93.5	83.9	48.4	87.1	77.4	25.8
Sonnet 4.5	90.3	83.9	58.1	93.5	80.6	29.0
Sonnet 4.6	93.5	83.9	83.9	87.1	87.1	32.3
Opus 4.8	87.1	83.9	90.3	83.9	80.6	41.9
Sonnet 5	87.1	87.1	80.6	90.3	87.1	45.2
<i>OpenAI</i>						
GPT-4o	96.8	51.6	61.3	90.3	74.2	16.1
GPT-4.1	83.9	51.6	41.9	80.6	64.5	19.4
GPT-5-nano	74.2	54.8	25.8	64.5	71.0	16.1
GPT-5	77.4	67.7	48.4	74.2	77.4	32.3
GPT-5.4-mini	80.6	45.2	54.8	87.1	80.6	12.9
GPT-5.4	80.6	83.9	64.5	77.4	77.4	9.7
GPT-5.6-Luna	83.9	77.4	35.5	80.6	74.2	6.5
GPT-5.6-Sol	77.4	80.6	32.3	71.0	80.6	12.9
GPT-5.6-Terra	64.5	80.6	35.5	71.0	80.6	12.9
Mean Δ				-2.3	+5.5	-32.0

Table 5: Context rot ablation accuracy (% , $n = 31$ per condition). Filtered = entry-relevant schemas only; Flood = 128 total schemas (relevant + corpus decoys). JSON = JSON tool calling; PTC = programmatic tool calling; Arg = filesystem-discovery. Bold indicates the highest value among the three paradigms within each condition. Anthropic model names omit “Claude” for space. Mean Δ = average change from filtered to flood across all 14 models. 95% Wilson confidence intervals range from $\pm 7.8\%$ to $\pm 16.6\%$ ($n = 31$; widest near 50% accuracy).

Above it, JSON tool calling exceeds programmatic tool calling as the response must enumerate all N tool-call objects. At $N = 30$, JSON tool calling uses 3,559 tokens versus 3,380 for programmatic tool calling. At $N = 48$, the gap widens to 5,097 versus 3,535.

4.4 Context Rot Ablation

The context rot ablation ($n = 31$ entries per condition) compares each paradigm under two context loads, *filtered* (only the entry’s relevant function schemas) and *flood* (128 total schemas, comprising the relevant functions plus corpus decoys from unrelated domains). Table 5 reports accuracy under both conditions.

JSON tool calling and programmatic tool calling are both stable under flooding. The mean accuracy change from filtered to flood is an absolute -2.3% for JSON tool calling and $+5.5\%$ for programmatic tool calling (individual model changes range from -6.5% to $+35.5\%$ absolute). The programmatic tool calling improvement under flood is driven by models that struggled with the strict type constraints of the filtered condition but found

the flood condition’s richer context helped them identify the correct function. This pattern is most visible in GPT-4.1 ($+12.9\%$ absolute), GPT-4o ($+22.6\%$ absolute), and GPT-5.4-mini ($+35.5\%$ absolute).

For reference, a filesystem-based internal condition degraded by an absolute 32.0% on average under flooding, where every model declined, consistent with context navigation being a structural weakness of file-based tool delivery (Sen et al., 2026a).

5 Analysis

5.1 Model Generation Predicts Programmatic tool calling Viability

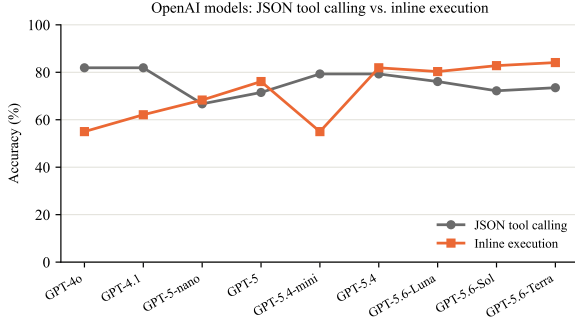
The clearest pattern in the results is that programmatic tool calling accuracy relative to JSON tool calling tracks model generation, not model family. All five Anthropic models match or exceed baseline across all three ablation studies, and so do the three newest GPT-5.6 variants. The models that fall below baseline under programmatic tool calling (GPT-4o, GPT-4.1, and GPT-5.4-mini) share a specific failure. They produce Python code with `\n` character sequences in place of real newlines, causing the subprocess to raise a syntax error on any multiline script.

The same system prompt, with the same newline instructions, produces correct multiline code from GPT-5-nano (a smaller and earlier model) while failing for GPT-5.4-mini (a larger and later model from a different training run), ruling out prompt configuration as the cause. We interpret this as a capability gap, though the specific cause is outside the scope of this paper.

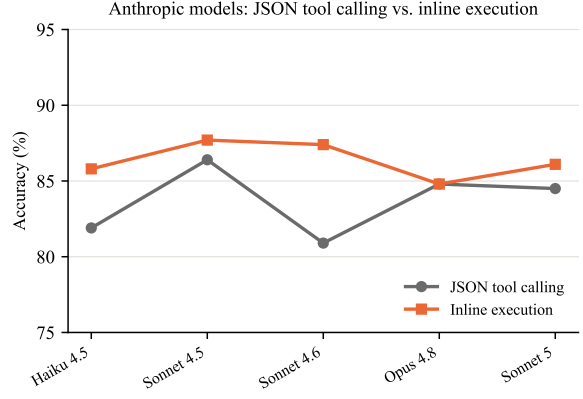
Figure 2 illustrates both trends side by side.

5.2 Programmatic tool calling Handles Fan-Out Structurally; JSON Tool Calling Does Not

Programmatic tool calling achieves near-ceiling accuracy for GPT-5 on the parallelism ablation, improving from 71.9% to 96.9% while JSON tool calling accuracy degrades above fan-out counts of 13. In JSON tool calling, the model must emit a parallel tool-call block in a single response, and at high fan-out it begins omitting calls. In programmatic tool calling, fan-out is expressed as a loop or a sequence of function calls in Python, which imposes no structural limit on count and benefits from the model’s code-generation training.



(a) OpenAI models. PTC starts 26.9% below JSON tool calling for GPT-4o and first crosses baseline at GPT-5-nano before converging fully by the GPT-5.6 generation.



(b) Anthropic models. PTC matches or exceeds JSON tool calling across all five models, with no generation-level gap.

Figure 2: Accuracy (%) on the BFCL v4 subset for JSON tool calling and programmatic tool calling (PTC) by model generation. The two families exhibit opposite patterns: OpenAI models required several generations to close the gap, while Anthropic models show consistent PTC parity from the outset.

The enumeration vs. aggregation split reveals a programmatic tool calling behavior that departs from the intended design. Several models produce the correct aggregation answer by drawing on parametric world knowledge rather than executing the enumeration calls. We report enumeration accuracy as the primary metric precisely because it measures whether the model actually invoked the tools, not just whether the final answer was correct.

To quantify the fan-out count at which this structural advantage emerges for Anthropic models, we probe Claude Sonnet 5 baseline at $N \in \{60, 70, 72, 75, 100\}$. Enumeration accuracy is 100% at $N \leq 70$, drops to 75% at $N = 72$, and reaches 0% at $N = 100$, placing the degradation onset between $N = 70$ and $N = 72$. Programmatic tool calling maintains 100% enumeration accuracy at both $N = 72$ and $N = 100$. This asymmetry does not appear in GPT-5.6-Sol, which holds 100% baseline enumeration accuracy through $N = 100$, suggesting the structural limit is specific to how Anthropic models serialize parallel tool-call blocks rather than a universal property of JSON tool calling.

5.3 Programmatic tool calling Reduces Latency on Chaining Tasks

Programmatic tool calling completes chaining entries in roughly half the wall-clock time of baseline for 13 of 14 models, with per-entry latency ratios ranging from 0.32 to 0.96 of baseline. JSON tool calling requires two inference turns for a sequential chain (one to call f_1 , one to receive its result and

call f_2), while programmatic tool calling resolves both calls in a single inference turn followed by one subprocess execution. GPT-5 is the exception, running at $2.8\times$ baseline latency under programmatic tool calling. Its extended reasoning output inflates generation time enough to erase the turn-reduction benefit.

6 Conclusion

As language model agents increasingly rely on tool use to act beyond their training data, the choice of tool interface (structured JSON calls versus programmatic tool calling) has practical consequences for chaining, parallelism, and robustness under real-world conditions. However, whether these advantages hold systematically across model generations and real-world task conditions on a standardized benchmark remained an open question. In this work, we presented a systematic evaluation of programmatic tool calling against native JSON tool calling across 14 language models on BFCL v4, using typed Python stubs as the tool interface. Programmatic tool calling matches or exceeds native JSON tool calling in 11 of 14 models, with the GPT-5.6 family achieving a 10.7% improvement over the JSON tool calling baseline. It matches or outperforms baseline in 13 of 14 models under parallel fan-out and holds stable under context rot conditions where baseline degrades 2.3% on average. These results demonstrate that programmatic tool calling is a viable and reliable alternative to JSON tool calling, with the remaining gap corre-

lating with model generation rather than model family.

7 Limitations

Four limitations bound the claims in this paper. First, BFCL v4 uses echo-return stubs. Each function returns its arguments verbatim rather than executing real API calls. This means we measure argument serialization accuracy, not end-to-end tool-use correctness. Results may not transfer to settings where return values affect downstream calls. Second, the ablation entry counts are small ($n = 31\text{--}52$ per condition), so individual model results carry wide confidence intervals and should be read as directional. Only the aggregate cross-model patterns (11 of 14 on BFCL v4, consistent programmatic tool calling improvement under flood) are large enough to interpret reliably. Third, a recent audit found 20% evaluator-human misalignment in BFCL v4’s LLM-judge evaluation mode (Vaghasiya et al., 2026). Our deterministic scorer avoids this path, but the benchmark’s ground-truth labels may still contain noise that our scorer inherits. Fourth, programmatic tool calling carries a fixed input-token overhead relative to JSON tool calling: the system prompt embeds the full instruction template as prose, whereas JSON tool calling passes function schemas through the API tools parameter, which some providers count separately from conversation input tokens. On the chaining ablation, programmatic tool calling uses $1.5\times$ the input tokens of JSON tool calling. This overhead reverses at high fan-out, as discussed in Section 4.3. Output token counts do not differ across paradigms.

References

- Tamer Alkhoul, Katerina Margatina, James Gung, Raphael Shu, Claudia Zaghi, Monica Sunkara, and Yi Zhang. 2025. [CONFETTI: Conversational function-calling evaluation through turn-level interactions](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, ACL 2025, pages 7993–8006.
- Anthropic. 2024. Building effective agents. <https://www.anthropic.com/research/building-effective-agents>.
- Kinjal Basu, Ibrahim Abdelaziz, Subhajit Chaudhury, Soham Dan, Maxwell Crouse, Asim Munawar, Sadhana Kumaravel, Vinod Muthusamy, Pavan Kapanipathi, and Luis A. Lastras. 2024. [API-BLEND: A comprehensive corpora for training and benchmarking API LLMs](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, ACL 2024, pages 12859–12870.
- Zehui Chen, Weihua Du, Wenwei Zhang, Kuikun Liu, Jiangning Liu, Miao Zheng, Jingming Zhuo, Songyang Zhang, Dahua Lin, Kai Chen, and Feng Zhao. 2024. [T-Eval: Evaluating the tool utilization capability of large language models step by step](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, ACL 2024, pages 9510–9529.
- Cloudflare. 2026. Code mode. <https://developers.cloudflare.com/agents/tools/codemode/>. Cloudflare Agents documentation. Last updated June 24, 2026.
- Yaxin Du, Yifan Zhou, Yujie Ge, Jiajun Wang, Xianghe Pang, Shuo Tang, Tuney Zheng, Bryan Dai, Jian Yang, and Siheng Chen. 2026. [HyperTool: Beyond step-wise tool calls for tool-augmented agents](#). *arXiv preprint arXiv:2606.13663*.
- Dongxin Guo, Jikun Wu, and Siu Ming Yiu. 2026. [The deterministic horizon: When extended reasoning fails and tool delegation becomes necessary](#). In *Proceedings of the 43rd International Conference on Machine Learning*, ICML 2026.
- Shijue Huang, Wanjuan Zhong, Jianqiao Lu, Qi Zhu, Jiahui Gao, Weiwen Liu, Yutai Hou, Xingshan Zeng, Yasheng Wang, Lifeng Shang, Xin Jiang, Ruifeng Xu, and Qun Liu. 2024. [Planning, creation, usage: Benchmarking LLMs for comprehensive tool utilization in real-world complex scenarios](#). In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 4363–4400.
- Hugging Face. 2024. smolagents: Building agents with code-first design. <https://huggingface.co/blog/smolagents>.
- Seth Karten, Alex L. Zhang, Kevin Thomas, Sebastian Müller, and Prime Intellect Team. 2026. Prime Agent: A self-improving RLM agent. <https://www.primeintellect.ai/blog/prime-agent>. Accessed: 2026-08-06.
- Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. [API-Bank: A comprehensive benchmark for tool-augmented LLMs](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, EMNLP 2023, pages 3102–3116.
- Tong Liu, Cheng Qian, Matej Cief, Yuan He, Daniele Dan, Nikolaos Aletras, and Gabriella Kazai. 2026. [On effectiveness and efficiency of agentic tool-calling and RL training](#). In *Proceedings of the 43rd International Conference on Machine Learning*, ICML 2026.
- Elias Lumer, Sahil Sen, Kevin Paul, and Vamse Kumar Subbiah. 2026. [Recursive agent harnesses](#). *arXiv preprint arXiv:2606.13643*.

Xuying Ning, Katherine Tieu, Dongqi Fu, Tianxin Wei, Zihao Li, Yuanchen Bei, Jiaru Zou, Mengting Ai, Zhining Liu, Ting-Wei Li, Lingjie Chen, Yanjun Zhao, Ke Yang, Bingxuan Li, Cheng Qian, Gaotang Li, Xiao Lin, Zhichen Zeng, Ruizhong Qiu, and 23 others. 2026. [Code as agent harness](#). *arXiv preprint arXiv:2605.18747*.

Sahil Sen, Akhil Kasturi, Elias Lumer, Anmol Gulati, and Vamse Kumar Subbiah. 2026a. [Is grep all you need? How agent harnesses reshape agentic search](#). *arXiv preprint arXiv:2605.15184*.

Sahil Sen, Elias Lumer, Anmol Gulati, and Vamse Kumar Subbiah. 2026b. [Chronos: Temporal-aware conversational agents with structured event retrieval for long-term memory](#). *arXiv preprint arXiv:2603.16862*.

Jay Vaghasiya, Vishvesh Bhat, Muhammad Ahmed Mohsin, and Asad Aali. 2026. [Benchmarking the benchmarks: A validity audit of tool-calling evaluation](#). *arXiv preprint arXiv:2607.02577*.

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024. [CodeAct: Executable code actions elicit better LLM agents](#). In *Proceedings of the 41st International Conference on Machine Learning, ICML 2024*.

Hong Yang, Qi Yu, and Travis Desell. 2026. [When does restricting a coding agent to execute_code help? A regime \$\times\$ agent-design ablation](#). In *Proceedings of the KDD 2026 Workshop on Software Engineering 3.0*.

Junjie Ye, Zhengyin Du, Xuesong Yao, Weijian Lin, Yufei Xu, Zehui Chen, Zaiyuan Wang, Sining Zhu, Zhiheng Xi, Siyu Yuan, Tao Gui, Qi Zhang, Xuanjing Huang, and Jiecao Chen. 2025. [ToolHop: A query-driven benchmark for multi-hop tool use](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics, ACL 2025*, pages 2995–3021.

A Per-Category Accuracy on BFCL v4

Category	<i>n</i>	JSON	$\pm$	PTC	$\pm$
<i>simple_python</i>	40	86.1	10.7	89.5	9.7
<i>multiple</i>	20	95.4	11.1	93.9	11.9
<i>parallel</i>	40	85.5	10.9	71.4	13.5
<i>par. multiple</i>	40	85.5	10.9	71.4	13.5
<i>live_simple</i>	25	76.6	15.9	76.3	15.9
<i>live_multiple</i>	96	73.1	8.7	83.0	7.5
<i>live_parallel</i>	24	66.1	17.7	56.8	18.4
<i>live par. mult.</i>	24	66.1	17.7	56.8	18.4
Overall	309	78.6	4.6	77.0	4.7

Table 6: Mean accuracy (%) per BFCL v4 category, averaged across all 14 models. JSON = JSON tool calling; PTC = programmatic tool calling. Means are macro-averaged over models. $\pm$ columns show 95% Wilson CI half-widths. Bold indicates the higher value per row. The *parallel* and *par. multiple* gaps are largely driven by the three OpenAI models with \n encoding failures.

B Programmatic tool calling Walkthroughs

This appendix shows concrete execution traces to illustrate how the programmatic tool calling agent loop operates in each evaluation setting. Each trace is a verbatim excerpt from a recorded trajectory, lightly condensed (absolute workspace paths are shortened to <workspace>/).

B.1 BFCL Single-Call (simple_python)

Task (*simple_python_0*): “Find the area of a triangle with a base of 10 units and height of 5 units.”

The stub module provided to the model contains:

```
def calculate_triangle_area(
    base: int, height: int,
    unit: str | None = None) -> dict:
    """Calculate the area of a triangle."""
    return _rpc.call('calculate_triangle_area',
                     base=base, height=height,
                     unit=unit)
```

The agent emits a single tool call:

```
execute_python(code="""
from stubs import calculate_triangle_area
result = calculate_triangle_area(
    base=10, height=5)
print(__import__('json').dumps(result))
""")
```

The subprocess prints:

```
{"calculate_triangle_area":
 {"base": 10, "height": 5}}
[exit code 0]
```

The stop middleware intercepts the next model call and terminates the loop. The scorer extracts function name and argument mapping from stdout, compares against ground truth, and records a single LLM turn for the entry.

B.2 Chaining Ablation

Task (*chaining_0*): “Compute the circumference of a circle with radius 7, then use that circumference as the side length of a square to find the square’s perimeter.”

The two relevant stubs are `geometry_circumference(radius)` and `geometry_square_perimeter(side)`.

Programmatic tool calling. The agent writes a single script that computes the intermediate value in Python before passing it to the second stub:

```
execute_python(code="""
import json, math
from stubs import (
    geometry_circumference,
    geometry_square_perimeter)

f1 = geometry_circumference(radius=7)
circ = 2 * math.pi * 7 # 43.982...

f2 = geometry_square_perimeter(side=circ)
print(json.dumps(f1))
print(json.dumps(f2))
""")
```

Subprocess output:

```
{"geometry.circumference": {"radius": 7}}
{"geometry.square_perimeter": {"side": 43.98}}
[exit code 0]
```

Both calls are resolved in a single subprocess, completing the entry in a single LLM turn.

B.3 Parallelism Ablation

Task (*fanout_population_7_a*): “Of these 7 countries — Zorbania, Keltross, Miventa, Drulan, Quelfor, Hartox, and Bivessa — which 3 have the largest population?”

The available stub is `country_info_population(country: str) -> dict`.

Programmatic tool calling. The agent issues one `execute_python` call that fans out all seven lookups concurrently and ranks the results in the same script:

```
execute_python(code="""
import asyncio, json
from stubs import country_info_population

pop = country_info_population # alias
async def main():
    cc = ["Zorbania", "Keltross", "Miventa",
          "Drulan", "Quelfor", "Hartox", "Bivessa"]
    res = await asyncio.gather(*[
        asyncio.to_thread(pop, country=c)
        for c in cc
    ])
    pairs = sorted(
        zip(cc, [r["population"] for r in res]),
        key=lambda x: x[1], reverse=True)
    top3 = [{"country": c, "population": p}
            for c, p in pairs[:3]]
```

```
print(json.dumps({"answer": top3}))
```

```
asyncio.run(main())
""")
```

Subprocess output:

```
{"answer": [
    {"country": "Bivessa", "population": 891234567},
    {"country": "Zorbania", "population": 847293441},
    {"country": "Miventa", "population": 523019876}]}
[exit code 0]
```

All seven stubs are invoked in parallel and Python sorts the results in the same script, completing the entry in a single LLM turn.

C System Prompts

JSON tool calling system prompt (condensed).

The JSON tool calling prompt instructs the model to call the provided tool definitions using the tool-calling API and to emit one JSON tool-call object per required function invocation, with no additional prose. Function schemas are passed as the `tools` parameter of the API request.

Programmatic tool calling system prompt (condensed).

The programmatic tool calling prompt embeds the source of a typed Python stub module and instructs the model to call `execute` exactly once with a `python3 -c '...'` command that imports the module, calls the required functions with the correct arguments, and prints results as JSON. The prompt specifies that real newlines (not `\n` escape sequences) must be used in multiline scripts. The stop middleware intercepts the first model call after the `execute` tool returns and terminates the agent loop without issuing a second inference.

Stub module design. Each stub function is generated from the benchmark’s function schema. It accepts the schema’s typed keyword arguments, immediately returns them as a structured dict, and prints the result to stdout. No external service is contacted. The scorer parses stdout to recover function name and argument mapping, then normalizes and compares against ground truth.